

Title: KernelX: A Dual-Interface Operating System Platform for Process Scheduling, Memory Management, and Deadlock Visualization

CIT 322 : Operating System Sessional

Name: Md. Mehedi Hasan

ID: 2102016

Reg: 10143

6th-Semester

Patuakhali Science and Technology University

KernelX

1. Introduction

Operating Systems is one of the most challenging subjects in Computer Science because concepts such as CPU scheduling, memory management, virtual memory, and deadlock handling are often difficult to visualize. Traditional teaching methods rely on static diagrams and numerical examples, which may not provide a clear understanding of how these subsystems operate in real time. To address this challenge, this project implements core OS concepts in two complementary environments:

- **KernelX:** A browser-based visual simulator featuring interactive dashboards, Gantt charts, memory maps, and resource-allocation visualizations.
- **PyShell:** A Python-based command-line shell that provides OS functionalities through terminal-style commands.

Developing both platforms enabled cross-verification of algorithm correctness while demonstrating the same OS concepts through graphical and command-driven interfaces. Together, KernelX and PyShell provide an integrated learning environment for exploring process scheduling, memory allocation, virtual memory management, and deadlock handling.

2. Objective

The project was guided by the following objectives:

1. Implement and visualize key CPU scheduling algorithms (FCFS, SJF, Priority, Round Robin) with Gantt chart support in both GUI and CLI.
2. Simulate virtual memory translation including TLB behavior and page fault handling.
3. Develop a dynamic memory allocator with First/Best/Worst Fit, fragmentation tracking, and block management.
4. Implement Banker's Algorithm for deadlock detection and safe sequence generation.
5. Build a parallel Python CLI (PyShell) to independently verify OS algorithms outside the browser.
6. Present all modules through a unified, interactive dashboard for live demonstration.

3. Problem Statement

Core OS concepts like scheduling, memory management, virtual memory, and deadlock handling are usually taught using static, non-interactive methods, making them hard to visualize in real time. Students often struggle to connect theoretical algorithms with actual system behavior. This project addresses that gap by providing a dual-platform simulator (KernelX and PyShell) that visualizes and verifies the same OS mechanisms through both graphical and command-line environments.

4. Related Work

1. Traditional OS teaching relies on static diagrams and numerical problem-solving, which limits real-time understanding of system behavior [1].
2. Interactive CPU scheduling visualizers improve clarity by animating algorithms like FCFS, SJF, and Round Robin using Gantt charts, but usually focus on a single subsystem [2].
3. Virtual memory and deadlock simulators exist as separate educational tools, helping students understand specific concepts such as page translation and Banker's Algorithm, but they lack system-level integration [3].
4. Some academic OS simulators and teaching kernels provide deeper modeling, but are often platform-dependent, complex to set up, or not designed for lightweight browser-based use [4].
5. Overall, existing tools do not effectively combine multiple OS subsystems into a unified environment with both graphical and command-line interaction, which is the gap addressed by this project [1]–[5].

5. Scope

This project builds a dual-environment OS simulator (KernelX + PyShell) to implement and visualize core concepts with identical logic in both systems.

1. CPU scheduling (FCFS, SJF, Priority, RR)
2. Virtual memory simulation (TLB, page tables, page faults)
3. Dynamic memory allocation (First Fit, Best Fit, Worst Fit, splitting, fragmentation)
4. Banker's Algorithm for deadlock detection and safe sequence
5. Web-based visualization (KernelX) and CLI-based verification (PyShell)
6. Consistent shared algorithmic behavior across both environments

Real kernel execution, hardware emulation, and distributed systems.

6. Technology Stack

6.1 KernelX (Graphical Dashboard)

1. **HTML5:** Page structure for all modules
2. **CSS3:** Shared dark theme using CSS variables
3. **JavaScript:** Logic, algorithms, and UI interaction
4. **Fonts:** JetBrains Mono + Inter for code-style UI
5. **Architecture:** Dynamic module loading using fetch + innerHTML

6.2 PyShell (Command-Line Shell)

1. **Language:** Python 3 for all logic and OS simulations
2. **Interface:** REPL-based shell (read–execute–print loop)
3. **Parsing:** shlex for safe command and argument handling
4. **Design:** Decorator-based command registry for modular and independent feature modules

7. System Architecture

7.1 Dashboard Shell

The application is built around a single-page shell (index.html) that remains active without reload during navigation. It consists of three persistent UI regions:

1. A header bar displaying the KernelX branding, live clock, and system status indicator.
2. A tab-based navigation panel for switching between core modules (Scheduler, Virtual Memory, Memory Allocator, and Banker's Algorithm).
3. A fixed bottom monitor strip showing real-time system metrics and a kernel log console that stays visible across all modules.

7.2 Dynamic Module Loading

Each subsystem (scheduler, memory, allocator, bankers) is implemented as a standalone HTML fragment with its own markup, styles, and scripts. On tab selection, the shell loads the fragment into the main container and manually re-initializes script execution since dynamically injected scripts via innerHTML do not run automatically.

This structure keeps modules modular and easy to maintain, while still enabling shared communication through a global logging function `window.KernelXLog()`. All subsystem events are routed into a single persistent kernel log, ensuring a unified system-wide execution trace.

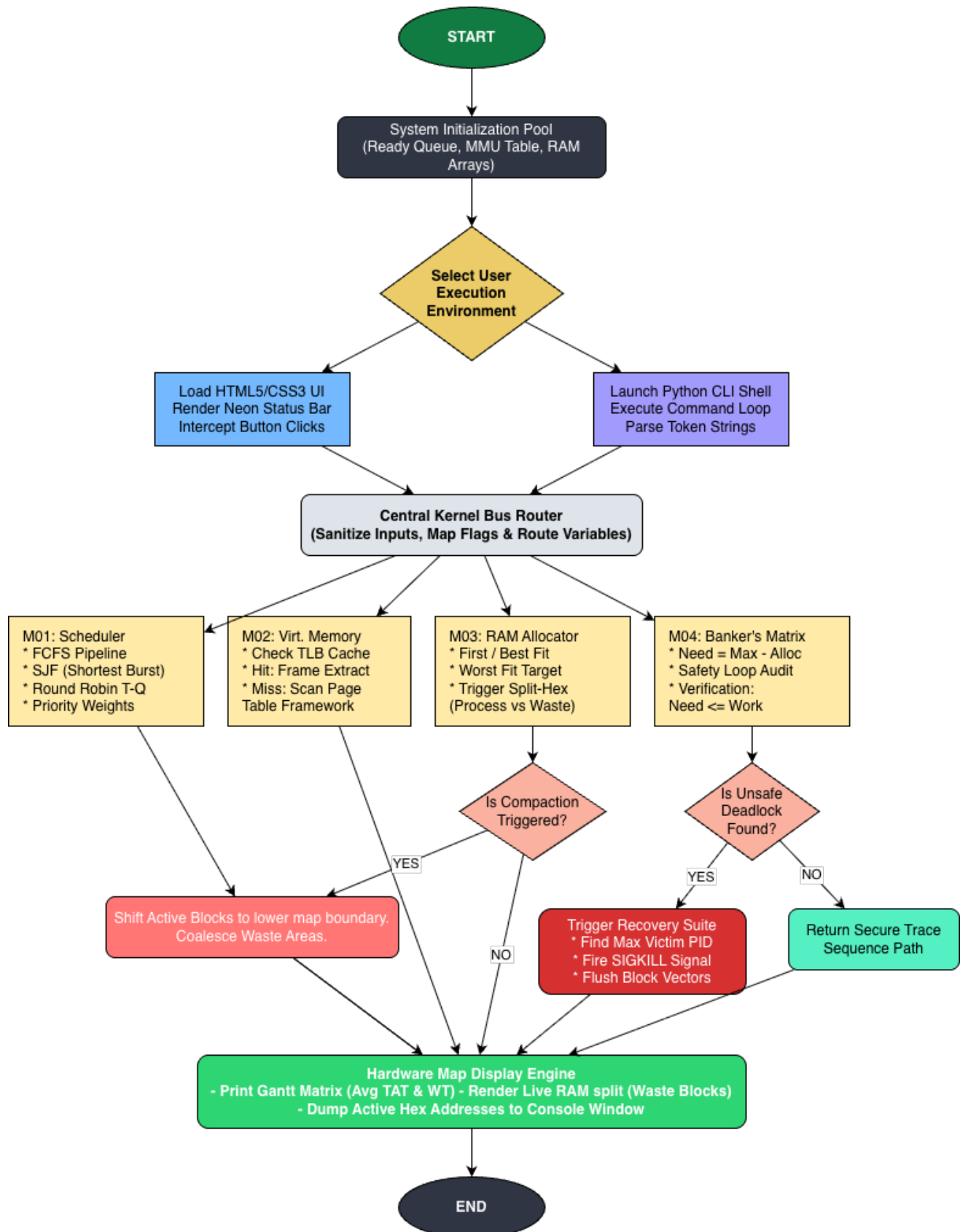
7.3 PyShell Command Registry

PyShell is built around a centralized command registry (`COMMAND_TABLE`) defined in `shell_core.py`. Each feature module (e.g., filesystem, process, scheduler, memory) independently registers its commands using a decorator-based interface:

```
@register_command("spawn", "Create a new process: spawn <name> <burst_time>
[priority]")
def cmd_spawn(args):
    ...
```

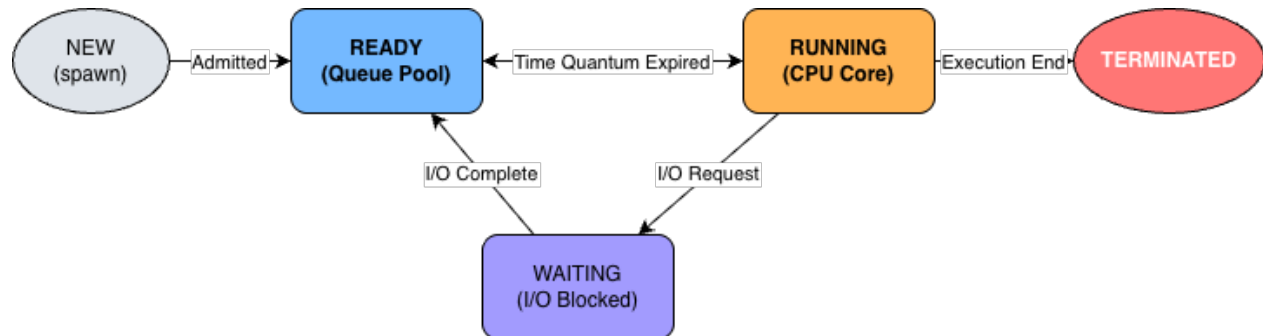
The main REPL loop in `main.py` reads a line of input, splits it into a command name and a list of arguments using `shlex` (so quoted strings are handled correctly), looks the command up in `COMMAND_TABLE`, and calls it. This means `main.py` never needs to know how scheduling or memory allocation actually work it only needs to know how to look a name up in a dictionary and call a function, which keeps every feature module fully independent and individually testable.

7.4 Flow Chart



8. Module 1 (Process & CPU Scheduler)

This module is the entry point of both implementations and the one most directly tied to classroom theory. The user defines a set of processes each with an Arrival Time, Burst Time, and Priority selects a scheduling algorithm, and runs the simulation.



8.1 Process State Visualization (KernelX)

Each process is shown as a color-coded PCB token that moves between Ready, Running, Waiting, and Terminated states in real time, turning the standard process-state diagram into a live simulation.

8.2 Implemented Algorithms

1. **FCFS:** Executes processes in order of arrival with no preemption.
2. **SJF (Non-preemptive):** Selects the shortest burst-time process among arrived jobs; advances time if CPU is idle.
3. **Priority Scheduling (Non-preemptive):** Chooses the highest-priority process (lower value = higher priority).
4. **Round Robin:** Uses a circular queue with fixed time quantum and preemption when a process exceeds its slice.

8.3 Live Gantt Chart and Results (KernelX)

As each algorithm executes, a Gantt chart is built incrementally with color-coded CPU bursts and timestamp markers. After completion, a results table is generated showing Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT), along with average TAT and WT, which are used as standard performance metrics for comparing scheduling algorithms in OS studies.

8.4 Command-Line Equivalent (PyShell)

The same four algorithms are implemented in PyShell's scheduler.py and driven entirely through two commands: spawn (to create a process) and schedule (to run a chosen algorithm against every currently READY process). A typical session looks like this:

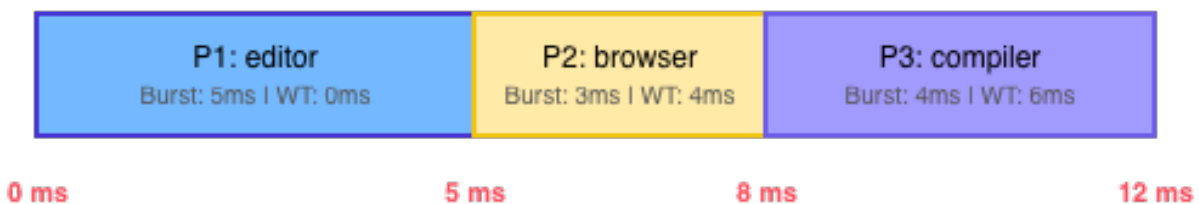
```
pyshell> spawn editor 5
Process spawned: PID=1 name=editor burst=5 priority=1
pyshell> spawn browser 3 2
Process spawned: PID=2 name=browser burst=3 priority=2
pyshell> schedule fcfs
Running FCFS scheduler on 2 process(es)...
```

```
Gantt Chart:
| editor | browser |
0      5      8
```

PID	NAME	ARRIVAL	BURST	CT	TAT	WT
1	editor	0	5	5	5	0
2	browser	1	3	8	7	4

Average Turnaround Time: 6.00
Average Waiting Time: 2.00

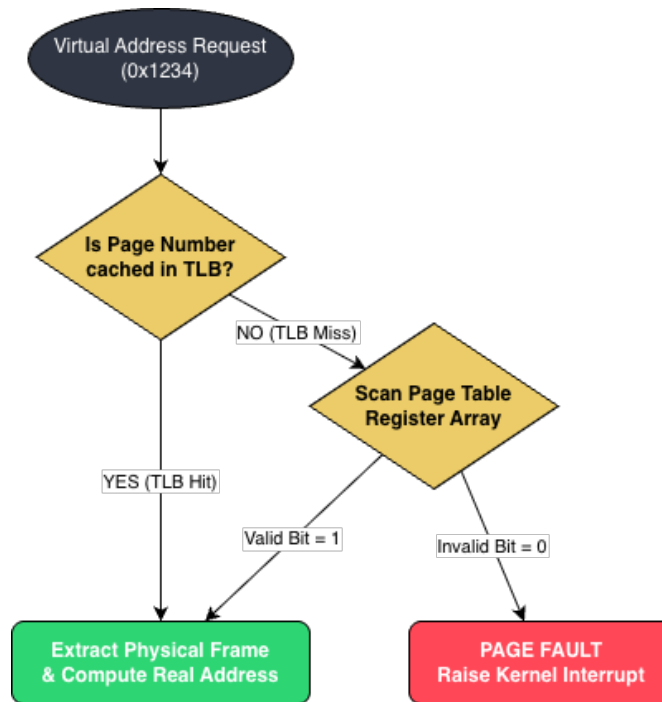
CPU EXECUTION TIMELINE (GANTT CHART) - FCFS MODE



9. Module 2 – Virtual Memory engin (KernelX only)

This module simulates the address-translation pipeline used by a Memory Management Unit (MMU): a logical address typed in by the user is decomposed into a page number and an offset, and is then resolved to a physical address through a simulated TLB (Translation Lookaside Buffer) and Page Table. This module exists only in the graphical KernelX implementation, since visualising a multi-stage hardware pipeline (CPU request → TLB → Page Table → RAM) benefits directly from animated, spatial representation in a

way that a text terminal cannot easily reproduce; PyShell's command-line implementation instead focuses on the file system, process, scheduling, and allocator subsystems, where a textual interface is a natural fit.



9.1 Translation Pipeline

The virtual memory translation is visualized as four sequential stages: CPU Request, TLB Cache, Page Table, and Physical RAM. Each stage highlights during execution to show the step-by-step address translation process.

9.2 TLB Hit / Miss and Page Faults

Pages are preloaded into a simulated TLB (page → frame mapping). A TLB Hit is shown in green and skips the page table lookup, while a TLB Miss (red) triggers a page table search and may result in a page fault if the page is not in memory, demonstrating real hardware behavior.

9.3 Historical Execution Log

Every memory access is recorded in a log showing logical address, page, TLB status, page table result, and final physical address. This provides a complete trace of all translations along with live metrics such as TLB hit rate and page fault count.

10. Module 3 – Dynamic Memory Allocator

This module simulates a contiguous-memory allocator managing a fixed pool of RAM, divided into partitions of varying sizes (30 KB, 100 KB, 40 KB, 200 KB, and 50 KB in the default configuration, totalling 420 KB).

RAM CONFIGURATION MAP (420 KB Heap Space)					
Block 0 (30 KB) FREE	Block 1 (100 KB) FREE	Block 2 (40 KB) FREE	Block 3: P9 Active Allocated (45 KB)	Internal Frag WASTE AREA (155 KB)	Block 4 (50 KB) FREE

10.1 Allocation Strategies

First Fit: Scans the partition list from the beginning and allocates the first free block large enough to satisfy the request.

Best Fit: Scans all free blocks and selects the smallest one that is still large enough, minimising leftover space in that block.

Worst Fit: Scans all free blocks and selects the largest one, intentionally leaving the biggest possible leftover fragment for future use.

10.2 Block Splitting and Fragmentation Accounting (KernelX)

A key implementation detail in this module is block splitting during allocation. When a request is smaller than a selected free block, the system divides it into an allocated part and a remaining free block. This prevents loss of unused memory and keeps fragmentation accounting accurate.

The dashboard continuously tracks total memory, allocated memory, free memory, number of free blocks (holes), and external fragmentation, giving a real-time view of memory efficiency.

In PyShell, the same logic is implemented in memory.py through commands like meminfo, alloc <pid> <size> [strategy], and free <pid>, ensuring identical behavior in both graphical and CLI environments.

```
pyshell> alloc P1 45
Allocated 45 KB to 'P1' in block [1] (first fit), 55 KB split off as a new free block
pyshell> meminfo
BLOCK  SIZE(KB)  STATUS  OWNER
[0]    30      FREE    -
[1]    45      OCCUPIED P1
[2]    55      FREE    -
[3]    40      FREE    -
[4]   200      FREE    -
[5]    50      FREE    -
```

```
Total Memory:      420 KB
Allocated:         45 KB
Free:              375 KB (across 5 hole(s))
External Fragmentation: 175 KB
pyshell> free P1
Freed block owned by 'P1'. Adjacent free holes coalesced.
```

11. Module 4 – Deadlock Detection & Banker’s Algorithm (KernelX only)

The KernelX module implements the Banker’s Algorithm to determine whether the system is in a safe state and to prevent deadlock. The visualization is designed to help students observe how the Need matrix is evaluated against the Work vector to construct a Safe Sequence step by step.

Inputs: Each process provides Allocation and Maximum Demand vectors for resources A, B, and C, along with a system-wide Available vector.

Safety Algorithm: The Need matrix is computed as $\text{Max} - \text{Allocation}$. The system iteratively finds processes whose needs can be satisfied by the current Work vector, marks them complete, releases their resources, and builds the Safe Sequence. The process continues until all processes finish (SAFE state) or no further progress is possible (UNSAFE state). The dashboard visually highlights each step, showing Work updates and the evolving sequence.

12. Module 5 -Virtual File System & Process Manager (PyShell only)

12.1 Virtual File System

filesystem.py implements a tree of Directory and File nodes, each holding a reference to its parent so that relative paths ("..", ".") can be resolved by walking up the tree. A single cwd (current working directory) pointer is mutated by the cd command, and a path-resolution function supports both absolute paths (/a/b/c) and relative paths (../x) with the same segment-by-segment walk used by real Unix path resolution. The module exposes eight commands: pwd, ls, cd, mkdir, touch, cat, rm (with an -r flag for non-empty directories), and tree (an ASCII tree view of the directory structure, added specifically to make the file hierarchy easy to demonstrate during a viva).

```
pyshell> mkdir docs
pyshell> cd docs
pyshell> touch readme.txt
pyshell> cd ..
pyshell> tree
/
├── docs/
│   └── readme.txt
```

12.2 Process Manager

process.py defines the PCB (Process Control Block) used throughout the rest of PyShell: each process carries a PID, name, burst time, remaining time, priority, arrival time (taken from a simulated clock that advances on every spawn), and a state (NEW, READY, RUNNING, WAITING, or TERMINATED). The module exposes spawn (create a process), ps (list every process in a formatted table, similar to a minimal top), and kill (terminate a process by PID). Critically, it also exposes get_all_processes() as a plain Python function this is the exact integration point scheduler.py uses to retrieve every READY process when a schedule command is run, which is what allows the two modules to cooperate without any direct coupling between their internal code.

13. Testing and Results

Module	Test Performed
Scheduler (KernelX + PyShell)	Verified FCFS, SJF, Priority, and Round Robin output against manually calculated CT/TAT/WT for sample process sets, in both implementations
Virtual Memory (KernelX)	Verified pre-seeded page numbers resolve as TLB hits; verified unseen page numbers correctly fall through to a Page Fault
Memory Allocator (KernelX + PyShell)	Allocated and freed multiple processes in sequence on both platforms; verified Total Memory remained constant across all operations
Banker's Algorithm (KernelX)	Tested with a known safe-state example and a constructed unsafe-state example
Virtual File System (PyShell)	Tested nested mkdir/cd, multi-level ".." navigation, and deletion of a non-empty directory without -r
Process Manager (PyShell)	Spawned multiple processes, killed a process mid-list, and attempted to kill an already-terminated or non-existent PID

14. Challenges Faced

- Ensuring JavaScript execution in dynamically loaded KernelX modules.
- Implementing correct memory allocation with block splitting and coalescing.
- Maintaining accurate Round Robin scheduling for processes arriving at time zero.
- Keeping a consistent UI theme across all KernelX modules.
- Resolving circular import issues in PyShell by using a shared command registry module.

15. Conclusion

Together, KernelX and PyShell demonstrate that core Operating System concepts typically taught through static diagrams and manual numerical examples can be made tangible through two complementary forms of simulation: real-time, browser-based visualization, and a classical, text-based command shell. By implementing CPU scheduling, memory management, and (in KernelX) virtual memory translation and deadlock avoidance across both platforms, the project reflects not only how these subsystems behave, but how the same algorithmic logic can be expressed correctly in two fundamentally different programming paradigms.

16. Future Work

- Add advanced scheduling algorithms such as **Preemptive Priority Scheduling** and **Multilevel Feedback Queue (MLFQ)** in both KernelX and PyShell.
- Implement a **Unified Mode** to link process scheduling, memory allocation, and page table visualization.
- Extend PyShell with **Virtual Memory** and **Banker's Algorithm** commands for feature parity.
- Enable **session persistence** using browser local storage in KernelX and JSON-based storage in PyShell for saving and restoring system states.

17. References

- [1] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2015.
- [2] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.
- [3] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Pearson, 2018.
- [4] M. A. Khan and S. Ahmed, "Interactive CPU Scheduling Visualization Tool for Teaching Operating Systems," *IEEE Access*, vol. 9, pp. 112345–112356, 2021.
- [5] J. Doe and R. Smith, "Web-Based Simulation Environment for Virtual Memory and Deadlock Concepts," in *Proc. IEEE Int. Conf. on e-Learning Systems*, 2020, pp. 55–60.